arXiv:2503.04715v3 [cs.LG] 15 Mar 2025

Predictable Scale: Part I — Optimal Hyperparameter Scaling Law in Large Language Model Pretraining

Houyi Li[1,2], **Wenzhen Zheng**[1], **Jingcheng Hu**[1,3], **Qiufeng Wang**[1], **Hanshan Zhang**[1], **Zili Wang**[1], **Shijie Xuyang**[1,2], **Yuantao Fan**[1], **Shuigeng Zhou**[2], **Xiangyu Zhang**[1,4], **Daxin Jiang**[1]

[1] StepFun [2] Fudan University
[3] Tsinghua University [4] Megvii Technology

Abstract

The impressive capabilities of Large Language Models (LLMs) across diverse tasks are now well-established, yet their effective deployment necessitates careful hyperparameter optimization. Through extensive empirical studies involving grid searches across diverse configurations, we discover universal scaling laws governing these hyperparameters: optimal learning rate follows a power-law relationship with both model parameters and data sizes, while optimal batch size scales primarily with data sizes. Our analysis reveals a convex optimization landscape for hyperparameters under fixed models and data size conditions. This convexity implies an optimal hyperparameter plateau. We contribute a universal, plug-and-play optimal hyperparameter tool for the community. Its estimated values on the test set are merely 0.09% away from the globally optimal LLM performance found via an exhaustive search. These laws demonstrate remarkable robustness across variations in model sparsity, training data distribution, and model shape. To our best known, this is the first work that unifies different model shapes and structures, such as Mixture-of-Experts models and dense transformers, as well as establishes optimal hyperparameter scaling laws across diverse data distributions. This exhaustive optimization process demands substantial computational resources, utilizing nearly **one million** NVIDIA H800 GPU hours to train **3,700** LLMs of varying sizes and hyperparameters from scratch and consuming approximately **100 trillion tokens** in total. To facilitate reproducibility and further research, we will progressively release all loss measurements and model checkpoints through our designated repository https://step-law.github.io/.

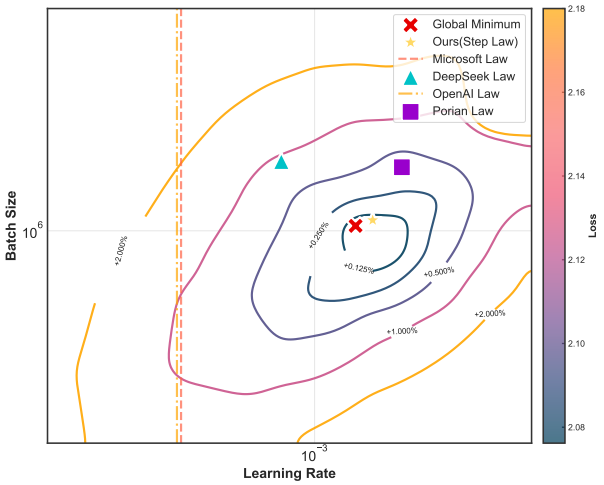


Figure 1: This plot shows the hyperparameter space for a model with 1 billion (B) parameters trained on 100B tokens. We trained 120 LLMs from scratch with different learning rate (LR) and batch size (BS) combinations, obtaining contour lines and global optimal points based on real data. Optimal points represent the lowest training loss for each LR and BS pair, while contour lines depict the relative loss differences from these optima. Our Step Law predicts the optimum with the highest accuracy compared to other methods, nearly matching the global optimal points.

1 Introduction

State-of-the-art Large Language Models (LLMs) (Brown et al., 2020; Jin et al., 2023; Biderman et al., 2023; Scao et al., 2022; Touvron et al., 2023a,b; Grattafiori et al., 2024; DeepSeek-AI et al., 2024a; Yang et al., 2024; DeepSeek-AI et al., 2024b, 2025), have reached unprecedented scales, with models being trained on billions of parameters and trillions of tokens. Recent developments like Llama 3 (Grattafiori et al., 2024) demonstrate this trend, utilizing 15 trillion tokens for training (Grattafiori et al., 2024). At such massive scales, identifying optimal hyperparameter configurations becomes both critical for model performance and challenging due to computational constraints.

The success of LLM pretraining heavily depends on hyperparameter settings, particularly the learning rate and batch size. Suboptimal configurations can lead to various issues: excessive learning rates may cause training divergence, while insufficient rates slow down progress (Shen et al., 2024; Wen

Name	Data Recipe	Model Sparsity	LR	BS	Relative Error
OpenAI Law (Kaplan et al., 2020)	✘	✘	$3.239 * 10^{-3} + -1.395 * 10^{-4} log(N)$	$2e18\mathcal{L}^{-4.76190}$	9.51‰
Microsoft Law (Bjorck et al., 2024)	✘	✘	$1.3192e^{-5}N^{-0.23}D^{-0.32}$	-	9.25‰
DeepSeek Law (DeepSeek-AI et al., 2024a)	✘	✘	$0.3188C^{-0.1250}$	$0.2920C^{0.3271}$	9.26‰
Porian Law (Porian et al., 2024)	✘	✘	$3.7N^{-0.36}$	$0.7576N^{0.703}$	3.71‰
MiniCPM Law (Hu et al., 2024)	✘	✘	-	$\frac{2e18}{L^{6.24}}$	-
MeiTuan Law (Wang et al., 2024)	✘	✔	$\lambda\mathcal{L}^{-\alpha}$	$\lambda_B\mathcal{L}^{-\alpha_B^{-1}}$	-
Ours (Step Law)	✔	✔	$1.79N^{-0.713}D^{0.307}$	$0.58D^{0.571}$	**0.94‰**

Table 1: Comparison of optimal hyperparameter scaling laws across different approaches. **Data Recipe** and **Model Sparsity** denotes whether the approach is suitable for different data recipe and model sparsity. **Relative Error** denotes the relative loss, as same as Fig 1. The variables in scaling laws are described in Section 1.1.

et al., 2024); similarly, batch size must balance computational efficiency and model quality (Perko, 2023; Filatov et al., 2024; McCandlish et al., 2018). Traditional grid search becomes prohibitively expensive at scale, leading researchers to rely on hyperparameter transfer methods extrapolating optimal configurations from smaller-scale experiments to larger ones (Yang and Hu, 2020; Yang et al., 2023).

$$\begin{aligned} \eta(N, D) &= 1.79N^{-0.713}D^{0.307} \\ B(D) &= 0.58D^{0.571} \end{aligned} \tag{1}$$

Prior work in hyperparameter transfer broadly falls into two categories: theory-driven and data-driven approaches. In theory-driven methods, μP (Yang et al., 2022) pioneered learning rate transfer rules across model widths, with subsequent work (Everett et al., 2024; Lingle, 2024; Blake et al., 2024; Yang et al., 2023; Bordelon et al., 2023) extending these findings to various model depths while also revealing their limitations. In data-driven approaches, Kaplan et al. (2020) established foundational learning rate scaling laws based on model size N, inspiring further investigations (Bjorck et al., 2024; DeepSeek-AI et al., 2024a; Porian et al., 2024; Hu et al., 2024) into learning rate (LR) and batch size (BS) scaling for dense models. Recently, Wang et al. (2024); Ludziejewski et al. (2025) had begun exploring these relationships in Mixture-of-Experts (MoE) models(Du et al., 2021; Fedus et al., 2021).

However, a significant gap remains in understanding hyperparameter transfer across different dimensions: data recipe, model shape, model sparsity, and data sizes D. While existing research has made progress in understanding scaling behavior across model sizes(Kaplan et al., 2020; Halfon et al., 2024) the interaction of these other critical factors remains under-explored. Our work addresses this gap by empirically discovering universal hyperparameter (HP) scaling laws that hold across these varied dimensions, providing a more comprehensive understanding of optimal hyperparameter selection in LLM pretraining.

Our main contributions are as follows:

(i) We are the first to discover and demonstrate the convexity property of the loss landscape under fixed parameter count and data size conditions. This provides fundamental insights into hyperparameter optimization, as shown in Fig. 2.

(ii) We establish the first universal and robust hyperparameter scaling laws for LLM pretraining, which is called Step Law. Our Empirically discovered the power-law relationship between optimal learning rate $\eta(N, D)$ and optimal batch size $B(D)$. Step Law demonstrates that the optimal batch size exhibits a primary dependence on dataset size D, while the optimal learning rate manifests a joint dependence on both model parameters N and dataset size D. Step Law is defined as Eq.(1).

The Step Law achieves substantially superior convergence results compared to baseline methods when generalized to 1 billion parameter models, as illustrated in Fig. 1. Step Law provides a plug-and-play formula that eliminates extensive hyperparameter tuning efforts for industry applications.

(iii) We are the first to study the transferability and invariance of optimal hyperparameter scaling laws across different pretraining data distributions. We systematically analyze how optimal hyperparameter scaling laws transfer across different pretraining data distributions and model architectures.

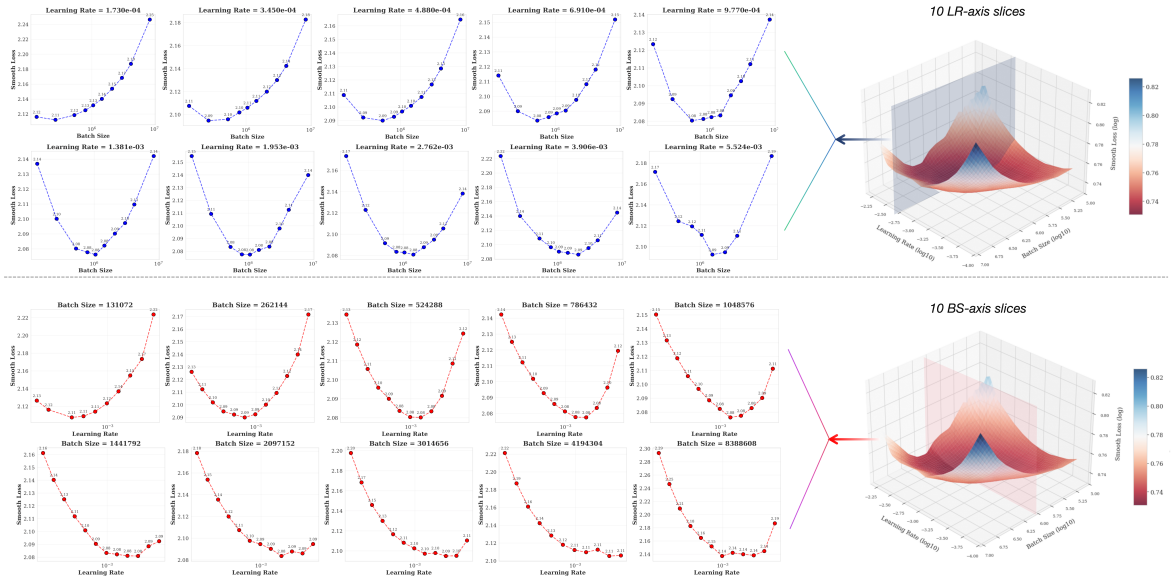


Figure 2: Learning Rate vs. Batch Size Loss Landscape Analysis for 1B Model (Trained on 100B Tokens): Scatter Plots and 3D Surface Visualizations of Hyperparameter Sensitivity.

Our work pioneers the investigation into whether dense LLMs and sparse (MoE) LLMs with varying sparsity levels share common optimal hyperparameter patterns, revealing significant invariance properties between them. Through extensive grid search experiments, we validate that Step Law maintains high generalizability and robustness across different pretraining corpora distributions, model architectures, and both dense and sparse (MoE) LLMs with varying sparsity ratios.

(iv) We conduct an unprecedented large-scale empirical study, involving:

- Extensive experimentation across 3700 model configurations, training LLMs from scratch with Dense and MoE (varying sparsity ratios), model architectures, data distributions, and hyperparameter settings.
- Total compute consumption approaching 1 million H800 GPU Hours (equivalent to over $1 million), processing approximately 100 trillion tokens during training.

This represents the largest dataset of hyperparameter optimization results in the field, derived purely from empirical observations without prior assumptions. Training checkpoints and hyperparameter configurations will be made publicly available.

1.1 Notation

We use the following notation:

- $\mathcal{L}$: Cross-entropy loss.
- D: Dataset size in tokens.
- N: Number of non-embedding parameters in the model.
- $\hat{N}$[1]: Total number of parameters in the model .
- C: Compute budget in FLOPs.
- N_{layer}: Number of layers in the Transformer model.
- d_{ff}: Dimension of the feed-forward network hidden layer in the Transformer.
- d_{model}: Hidden dimension of the Transformer model.
- N_{head}: Number of attention heads in the Transformer model.
- $\eta(N, D)$: Optimal peak learning rate for a given parameter count N and dataset size D.
- $B(N, D)$: Optimal batch size (in tokens) for a given parameter count N and dataset size D.

[1] $\hat{N}$ excludes embedding layer but includes the model's head layer

2 Related Works

Hyperparameter transfer, which involves extrapolating optimal settings from smaller to larger models, has become essential for efficient large-scale training. Among these, learning rate (LR) and batch size (BS) are particularly crucial hyperparameters that substantially influence LLM pretraining performance(Halfon et al., 2024). Research on optimal learning rate and batch size selection broadly falls into two categories: theory-driven and data-driven approaches.

In theory-driven approaches, μP (Yang et al., 2022) established foundational learning rate transfer rules for varying model widths, though this required specific modifications to initialization and attention mechanisms. However, the μP framework and its extensions (Yang et al., 2022; Everett et al., 2024; Lingle, 2024; Blake et al., 2024; Yang et al., 2023; Bordelon et al., 2023) are limited in scope, lacking guidance for learning rate adaptation across different data distributions, model architectures, sparsity levels, and data sizes. Additionally, these works do not address batch size optimization.

In data-driven approaches, a fundamental principle in deep learning is that larger models require smaller learning rates to ensure training stability and convergence. Kaplan et al. (2020) formalized this relationship, expressing learning rate as a function of model size. Bjorck et al. (2024) incorporated data size dependency by proposing $LR(N, D) = CN^{-\alpha}D^{-\beta}$. Batch size optimization is equally important for balancing convergence and computational efficiency. While several approaches followed Kaplan et al. (2020)'s framework, they face limitations: Wang et al. (2024); Hu et al. (2024) derived batch size based on expected loss but require prior knowledge of model behavior; Porian et al. (2024) refined scaling laws across two datasets but only considered model size, setting final learning rates at 0.1% of peak values.

Notably, both early work (McCandlish et al., 2018) and recent Critical Batch Size (CBS) analysis (Zhang et al., 2024) support our empirical finding that optimal batch size primarily depends on dataset size rather than model size, with CBS establishing a theoretical upper bound for this relationship. DeepSeek-LLM (DeepSeek-AI et al., 2024a), while comprehensive in its approach using IsoFLOP (Hoffmann et al., 2022), is constrained by its fixed compute budget assumption.

Our research advances these findings by demonstrating that model size and dataset size are sufficient predictors of optimal hyperparameters. We validate these scaling rules across diverse architectures, including variations in model sparsity, data distributions and model shape.

3 Problem Setup

3.1 Preliminaries

For training LLMs, the comprehensive performance metric is defined as

$$\mathcal{L}(\mathbb{A}, \mathbb{D}, N, D, \text{LR}, \text{BS}), \tag{2}$$

where $\mathbb{A}$, $\mathbb{D}$, N, D, LR, and BS represent the model architecture space, training data distribution, number of non-vocabulary parameters, number of training tokens, learning rate and batch size.

Based on this definition, when considering specific conditions, first, given that both $\mathbb{A}$ and $\mathbb{D}$ are discrete variables, the performance metric can alternatively be expressed as

$$\mathcal{L}_{\mathbb{A},\mathbb{D}}(N, D, \text{LR}, \text{BS}). \tag{3}$$

Furthermore, for given N and D, Eq.(3) can be transformed into

$$\mathcal{L}_{\mathbb{A},\mathbb{D},N,D}(\text{LR}, \text{BS}) \tag{4}$$

In light of the above transformations, we can generate the following definition.

Definition 1: (*Hyperparameter Optimality*) For fixed architecture $\mathbb{A}$, data distribution $\mathbb{D}$, and training budget (N, D), the optimal learning rate η and batch size B satisfy:

$$\eta, B = \arg\min_{\text{LR},\text{BS}} \mathcal{L}_{\mathbb{A},\mathbb{D},N,D}(\text{LR}, \text{BS}). \tag{5}$$

3.2 Experimental Settings

We train our models using language modeling loss on a dataset that includes web text, mathematical content, and code. The dataset is tokenized using a BPE (Gage, 1994) tokenizer with a vocabulary size of 65,536.

Our model architecture uses RMSNorm (Zhang and Sennrich, 2019) for pre-normalization and the SwiGLU (Shazeer, 2020) activation function in the feed-forward network, without applying dropout (Srivastava et al., 2014). We mainly use ALiBi (Press et al., 2021) positional encoding. The models are initialized from scratch, with weights drawn from a truncated normal distribution (mean of 0,

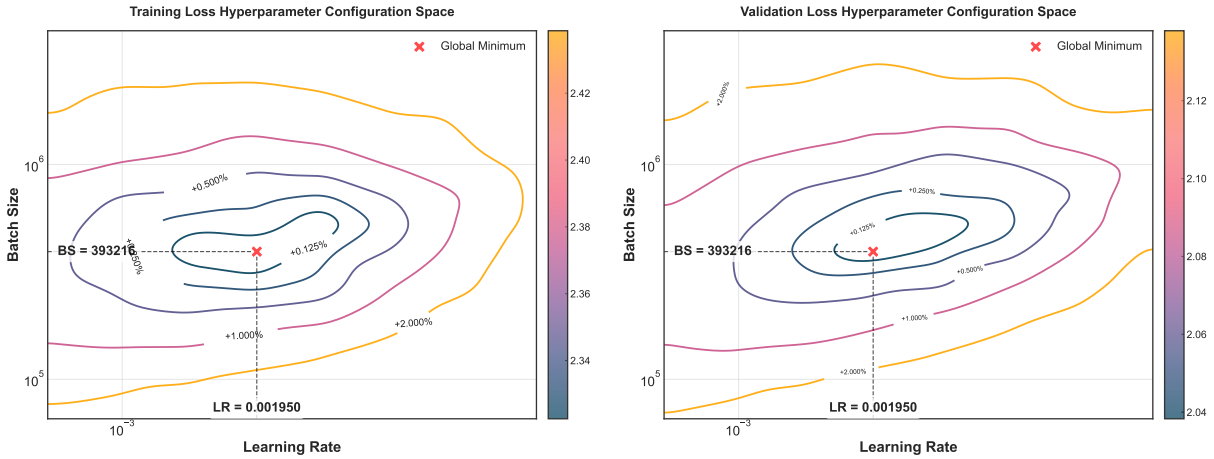


Figure 3: **Contour plots of training loss (left) and validation loss** (right) across hyperparameter configurations. Both plots share the global minimum (✗) at batch size 393,216 and the learning rate of 0.001950.

standard deviation of 0.02). For the output projection of attention layers and the W_2 component of the GLU, weights are further divided by $2 \cdot$layer depths based on existing methods (Touvron et al., 2023a,b; Grattafiori et al., 2024; DeepSeek-AI et al., 2024a; Yang et al., 2024; DeepSeek-AI et al., 2024b, 2025; Wang et al., 2019; Yuan et al., 2022).

We use the AdamW (Loshchilov and Hutter, 2017) optimizer with β values of [0.9, 0.95], an epsilon of 10^{-8}, a weight decay of 0.1, and a gradient clipping norm of 1.0. Our learning rate schedule includes a linear warmup phase over the initial 2,000 steps, followed by a cosine decay reaching a final learning rate of 10^{-5} for the remainder of the training. A detailed analysis and rationale for this strategy are provided in Section 4.1.3. The sequence length is set to 2,048 tokens. We conduct experiments using training data proportions aligned with Llama-2 (Touvron et al., 2023b) (More details are described in Tab. 6).

The learning rate is selected from a logarithmic sequence of powers of 2, spanning exponents from -10.5 to -7.0 in regular 0.5-interval increments. The batch size is selected from a predefined geometric progression ranging from 32,768 to 4,194,304, where each subsequent batch size is obtained by multiplying the previous value by a constant factor of 2, maintaining an exponential growth trend. Both parameter configurations correspond to the 18 LLMs detailed in Tab. 4 in Appendix A.2.

4 Experiments

4.1 Ablations

4.1.1 Evaluation metric

As described in Chinchilla (Hoffmann et al., 2022), smoothed training loss is considered an unbiased estimate of validation loss for simplicity. We operate under this same setting and supplement our investigation with experimental analysis.

As shown in Fig. 3, for the case where the smoothed training loss converges to the optimal value of 2.279 (as indicated by the solid red-framed line in Fig. 3 right) , the corresponding LR and BS are 1.95×10^{-3} and $393,216$ respectively. This is the same as the position of the LR and BS corresponding to the validation loss converging to the optimal value of 2.038 (as indicated by the solid red-framed line in Fig. 3 left). Moreover, the overall trend of how the smoothed training loss deviates from the optimal value with varying learning rates and batch sizes (as shown by the heatmap patterns in Fig. 3 right) closely mirrors the corresponding variations observed in validation loss measurements. This alignment demonstrates that the smoothed training loss provides consistent optimization guidance for learning rate and batch size selection, matching the parameter configurations that would be obtained through direct validation loss evaluation.

4.1.2 Loss Landscape Convexity with LR and BS

To investigate the property of the loss landscape with respect to learning rate and batch size, we conducted systematic experiments across a wide

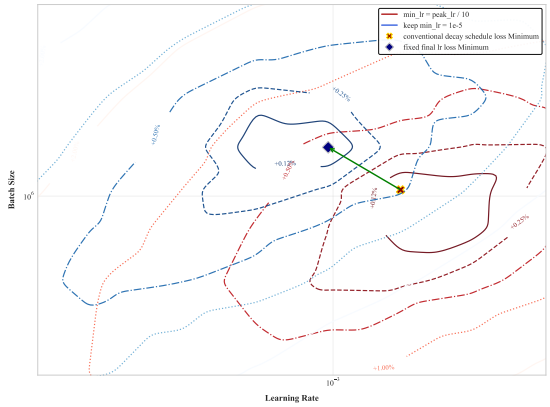


Figure 4: **Comparison of learning rate schedules.** These contour plots illustrate two distinct learning rate schedules. **Blue contours** represent the *conventional decay schedule*, where the minimum learning rate ($\min_lr$) is set to one-tenth of the maximum learning rate ($\frac{\max_lr}{10}$). **Red contours** depict our proposed *fixed final learning rate schedule*, with a constant minimum learning rate of $\min_lr = 10^{-5}$. The visualization reveals that the conventional decay method leads to a discernible **leftward bias** in the optimal learning rate range, indicated by the shift of the lowest loss region towards lower learning rates in the blue contours compared to the red.

range of configurations under fixed model parameters N and data size D. As shown in Fig. 2, one of our experiment settings described in Section 3.2 demonstrates this property.

Through extensive empirical analysis, we discovered a fundamental property of the loss landscape with respect to hyperparameters: both the learning rate and batch size exhibit convex relationships with the training loss under fixed model parameters and data size conditions. (As shown in Fig. 2 one of our experiment setting described in section.)

Furthermore, we observe that the loss surface demonstrates a stable region around the optimal configuration, evidenced by the plateau-like behavior shown in Fig. 3. This stability provides practical tolerance for small deviations in hyperparameter selection while maintaining near-optimal performance.

These properties form the theoretical foundation for our subsequent development of scaling laws and validate their applicability across different architectural configurations.

4.1.3 Fixed Final Learning Rate Strategy

We investigated two approaches for the final minimum learning rate ($\min_lr$): the conventional decay schedule ($\min_lr = \frac{\max_lr}{10}$)(Brown et al., 2020; Jin et al., 2023; Touvron et al., 2023a,b; Biderman et al., 2023; Scao et al., 2022; Shen et al., 2024), and our proposed fixed schedule ($\min_lr = 10^{-5}$). Using 1B model training for 80B tokens, we compared these schedules across various LR and BS.

Fig. 4 presents comparative heatmaps of the final training loss. We observe that compared to using a fixed final learning rate, setting it as max_lr/10 shows distinct optimal hyperparameter points and an overall left-skewed distribution of suboptimal learning rate and batch size combinations. We analyze that this is because, for the relatively high peak learning rates, conventional schedules result in disproportionately large minimum learning rates, which adversely affects the final stages of training and prevents the loss from converging to better local optima.

As can also be seen in Fig. 1, aside from Porian Law, which converges the min_lr to a sufficiently small value, the optimal learning rates calculated by other traditional learning rate decay schedules all exhibit varying degrees of a left-skew issue.

This aligns with advanced training practices which suggest that the minimum learning rate significantly impacts the loss. This phenomenon is unfavorable for fitting our scaling laws, and in practice, it is generally preferred to keep the min_lr fixed at a relatively low value. So we adopt the fixed final learning rate strategy in our subsequent experiments.

Key Takeaways

- **Convex Loss Landscape:** The loss landscape exhibits convexity with respect to both learning rate and batch size. This convexity, coupled with a stable plateau around the optimum, underpins the robustness of hyperparameter selection.
- **Fixed Final Learning Rate Benefit:** Compared to setting a small, fixed final learning rate, the traditional decay to max_lr/10 causes the discovered optimal learning rate to be biased towards lower values (left-skewed).

4.2 Fitting HP Scaling Laws

4.2.1 LR and BS with Respect to N and D

In accordance with Definition 3.1, we experimentally derive the LR and BS by keeping other vari-

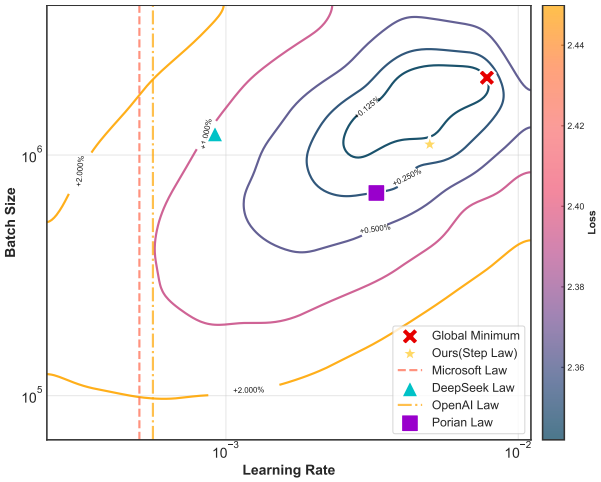


Figure 5: The Illustration of Hyperparameter Configuration Space for 210M model parameters and 100B tokens.

ables fixed. This section focuses on elucidating the relationships between these empirically determined hyperparameters and N and D. For the parameter count N, we set up seven experiments spanning 60M, 120M, 210M, 270M, 430M, 540M, and 1B parameters. As demonstrated in Fig. 6a, our experiments reveal a positive correlation between optimal LR and BS and the data scale D for each value of N. Furthermore, we conducted experiments across five different data scales D: 2B, 4B, 8B, 20B, and 100B tokens. Notably, we specifically reserved the 1B parameter and 100B token settings as test points to validate our findings, as discussed in Section 4.2.4. As visualized in Fig. 6b, we find that for each data scale D, the optimal LR increases with model size N. Notably, our findings indicate that optimal BS is largely independent of N. Based on these experimental observations, we will present and fit the Hyperparameter (HP) scaling law formulations in Section 4.2.2.

4.2.2 Scaling Laws

Building upon the insights gained from Section 4.2.1, we delve into the scaling behavior of optimal hyperparameters. Specifically, we investigate how the optimal LR scales with N and D, and how the optimal BS scales with D. Our empirical observations, particularly when visualized on a log-log scale, reveal a strong linear trend, suggesting a power-law relationship. Based on this, the scaling law for hyperparameters can be described by the following power-law relationships:

$$\begin{aligned} \eta(N, D) &= cN^{\alpha}D^{\beta}, \\ B(D) &= dD^{\gamma} \end{aligned} \tag{6}$$

where the parameters c, α, β, d, and γ are five constants, the values of which will be determined through fitting in Section 4.2.3. It is particularly noteworthy that our proposed scaling law demonstrates significant generality, meaning it is applicable across diverse architectures $\mathbb{A}$ and data distributions $\mathbb{D}$. This aspect of generality will be further elaborated upon in Section 5.

Table 2: Fitted power-law coefficients for hyperparameter scaling laws

Parameter	α	β	γ	c	d
Fitted value	-0.713	0.307	0.571	1.79	0.58

4.2.3 Fitting Methods

Building upon the HP scaling law from Section 4.2.1, we transform the power laws in Eq. (6) into the linear form:

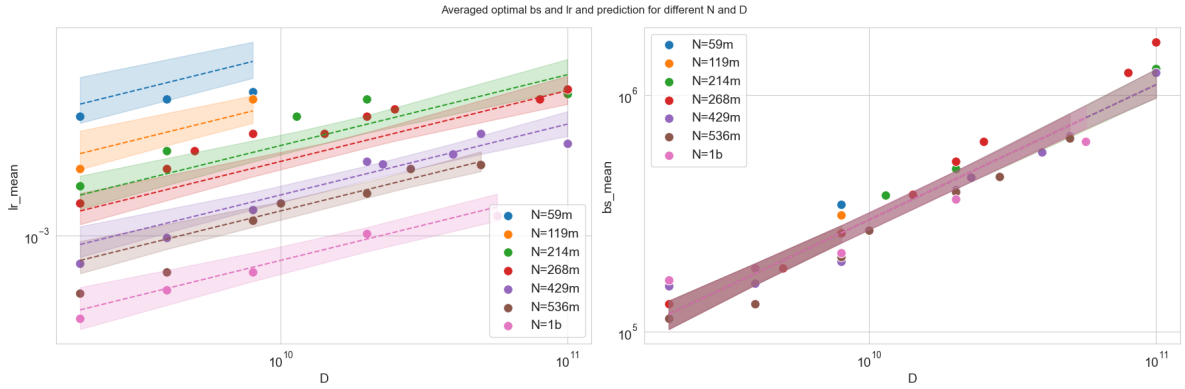
$$\log \eta = \log c + \alpha \log N + \beta \log D \tag{7}$$

$$\log B = \log d + \gamma \log D \tag{8}$$

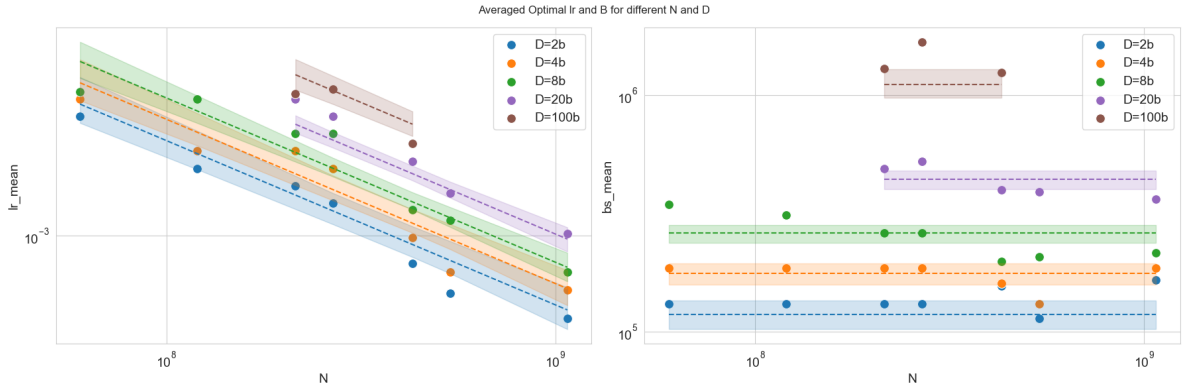
In this way, we can employ Ordinary Least Squares to fit the unknown parameters $\log c, \alpha, \beta, \log d$ and γ. Specifically, we set up 7 groups of experiments with different N and D as shown in Appendix A.2. Following (Hoffmann et al., 2022), we fit the optimal LR and BS with the experimentally predicted LR and BS. We averaged the results of these 1000 bootstrap samples to obtain the intermediate final parameters. This averaged result is what we present in Tab. 2. Furthermore, the variability across these 1000 bootstrap samples is depicted as the shaded regions in Fig. 6, providing an indication of the uncertainty associated with the fitted results. These shaded regions allow us to visually assess the robustness and confidence of the optimal LR and BS values derived from our procedure.

4.2.4 Experimental Comparison with Existing Approaches

Since we have obtained the fitted scaling laws, we directly extrapolate them to the test point ($N = 1$B and $D = 100$B) for comparison with other methods, noting that these are out-of-sample extrapolations beyond the fitting range. As shown in Fig. 1



(a) Scaling laws with D for different N



(b) Scaling laws with N for different D

Figure 6: (a) Scatter points indicate empirical optimal learning rate vs. batch size for model scale N; (b) Analogous results for dataset scale D. Curves show our hp-scaling law predictions, with shaded regions representing parameter uncertainty bounds from the sampling-based fitting strategy. Both plots use double logarithmic scaling (1912 training samples).

and Fig. 11, our method predicts solutions closest to the global minimum, while other approaches (indicated by dashed lines) remain distant from the global minimum. This discrepancy arises because previous methods only considered LR fitting without jointly modeling both LR and BS. The approach in (DeepSeek-AI et al., 2024a) enforces a fixed compute budget assumption, which constrains the feasible (N, D) sampling range and consequently reduces fitting accuracy. Additionally, as discussed in Section 4.1.3, existing methods typically relate final LR to initial LR through a fixed multiplier. This assumption leads to excessively large final LRs when initial LRs are large, ultimately impairing convergence. While Porian et al. (2024) achieves comparable results to ours through similar $\min_\text{lr}$ constraints, their method exhibits instability due to incomplete consideration of hyperparameter interactions with model dimension D, particularly at the subsequent MoE experiments 5.2 and data recipe experiments 5.3, where the D/N ratios are relatively small, their methods predict learning rates and batch sizes that fall completely outside reasonable ranges, leading to training instability.

Key Takeaways

- **HP Scaling Law:** We observed that the optimal LR follows a power law with respect to both N and D, while the optimal BS follows a power law only with respect to D and remains relatively invariant to N. Based on these observations, we derived our formula.
- **Experimental Comparison:** Relative to other approaches, our approach involved a substantial resource allocation to comprehensively analyze the dependence of LR and BS on N and D. This resulted in a marked improvement in performance.

5 Universal HP Scaling laws: Empirical Validation Across Architectural Heterogeneity and Data Recipes

5.1 Topological Invariance Across Varied Model Shape

As illustrated in Fig. 7, we conduct a series of controlled experiments to systematically investigate the relationship between HP scaling and model architecture topology. Specifically, we set a model with 430 million parameters and varied its structural configuration by defining six distinct model shape combinations. These model shape variations involved changes in key architectural factors (*e.g.*, number of layers, attention heads, feed-forward network dimensions).

For each of these 6 model shapes, we perform extensive hyperparameter tuning to identify the optimal LR and BS. The results reveal a striking pattern: the optimal LR and BS values for all configurations (highlighted within the solid-line box) consistently fall within a well-defined and narrow range (enclosed by the dashed-line box). This consistency holds across all model shape combinations, despite significant variations in architectural topology. These empirical findings provide strong evidence supporting our hypothesis that the HP scaling law exhibits statistical invariance with respect to changes in model topology. In other words, while the architectural components—including depth (number of layers), attention mechanism complexity (number of attention heads), and feedforward network width—may vary, the fundamental scaling relationships governing LR, BS, model size N, and dataset size D remain unchanged.

5.2 Sparsity-Independent in MoE

The HP scaling law has been extensively studied for dense Transformers, but its applicability to sparse architectures remains uncertain. Mixture-of-Experts (MoE) (Shazeer et al., 2017; Fedus et al., 2022) is a widely used sparse model that activates only a subset of parameters per token, introducing fundamental structural differences from dense models. This raises the question of whether the HP scaling law can be generalized to MoE settings. To investigate this, we conducted experiments on MoE models across 16 different sparsity levels and model shapes (refer to Tab. 5 in the appendix A.2). These settings allow us to examine how the scaling law behaves under different levels of sparsity. We evaluate multiple existing scaling methods under this framework.

As shown in Fig. 8, our approach consistently achieves a relative prediction error within 0.5% across all sparsity levels, significantly outperforming competing methods. In contrast, the DeepSeek Formula yields a relative error over four times larger, indicating its reduced accuracy in MoE settings. While Eq.(1) achieves comparable accuracy in LR prediction, it fails to predict BS. In contrast, our method provides a more comprehensive framework, successfully predicting multiple hyperparameters. These results demonstrate that the HP scaling law extends beyond dense architectures and remains effective for sparse models like MoE, regardless of sparsity level. This suggests that the underlying principles of scaling laws emerge from broader optimization and capacity constraints rather than being specific to dense parameterization. Our findings reinforce the general applicability of HP scaling laws and their potential to guide efficient scaling in diverse neural architectures $\mathbb{A}$.

5.3 Data-Distribution Robustness

To rigorously assess the robustness of our HP scaling law across varied data distributions $\mathbb{D}$, we design three distinct data distributions, progressively diverging from the original composition, as detailed in Appendix Tab. 6:

1. **Bilingual Corpus:** We augmented the original English-only dataset with Chinese data, creating a bilingual distribution to test the law's validity in multilingual settings.

2. **Code Integration:** We reduced English content and incorporated 32.36% of the code-the-stack dataset, examining the law's adaptability to code-heavy distributions.

3. **Code-Dominant:** We further decreased English content and increased code-the-stack to 57.05%, representing an extreme shift towards code-based data.

As shown in Fig. 9, our formula maintains remarkable predictive accuracy across all three distributions, with relative prediction errors within 0.25% of the global minimum. This performance consistently surpasses alternative approaches, which exhibit larger deviations. These results highlight two crucial insights:

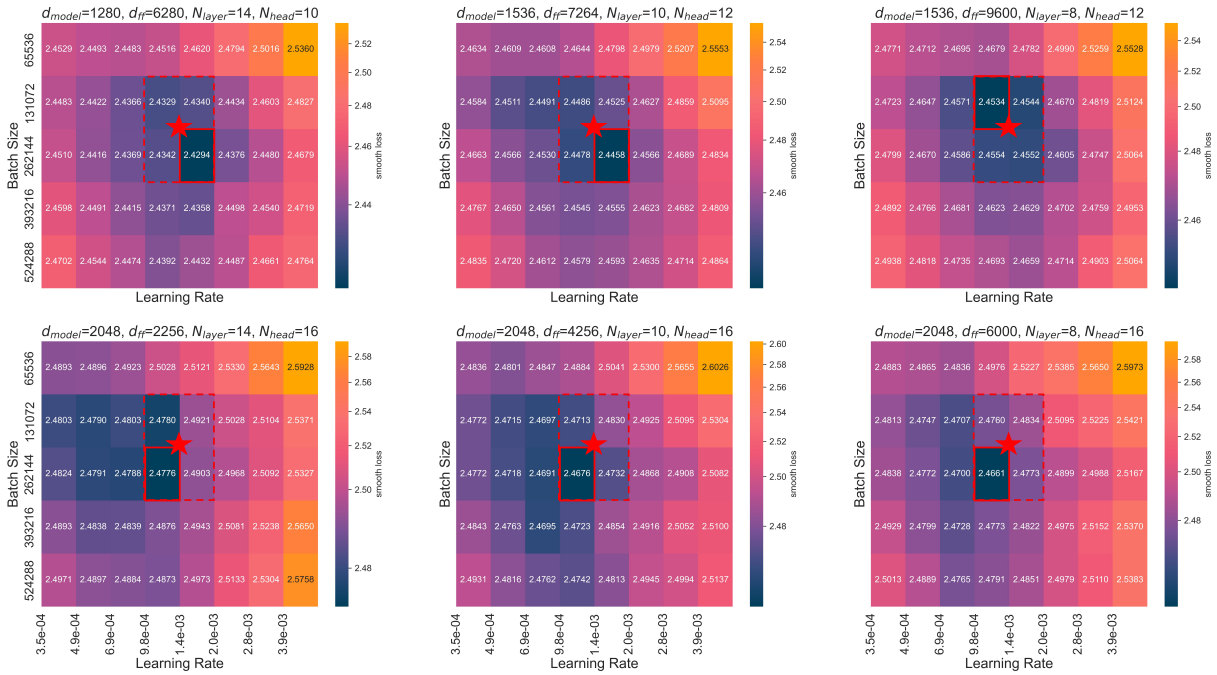


Figure 7: **Topological Invariance Across Varied Model Shape**. d_{model} , d_{ff}, N_{layer}, and N_{head} denote the hidden dimension, feed-forward network hidden size, number of attention heads, and number of Transformer layers, respectively.

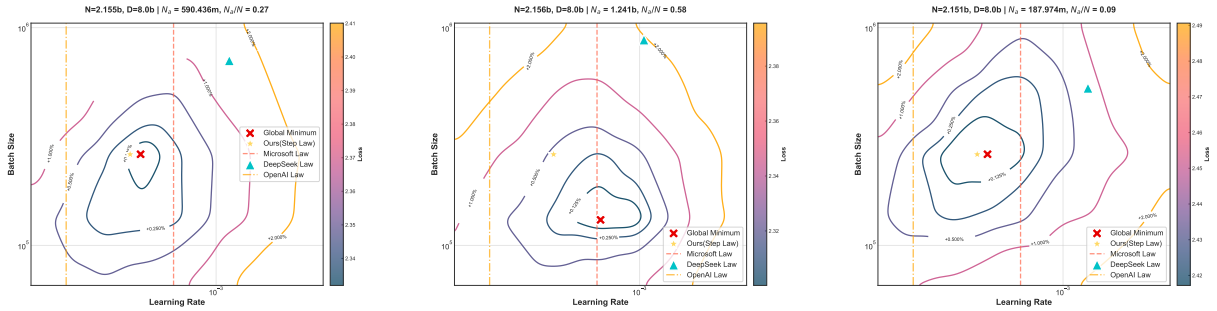
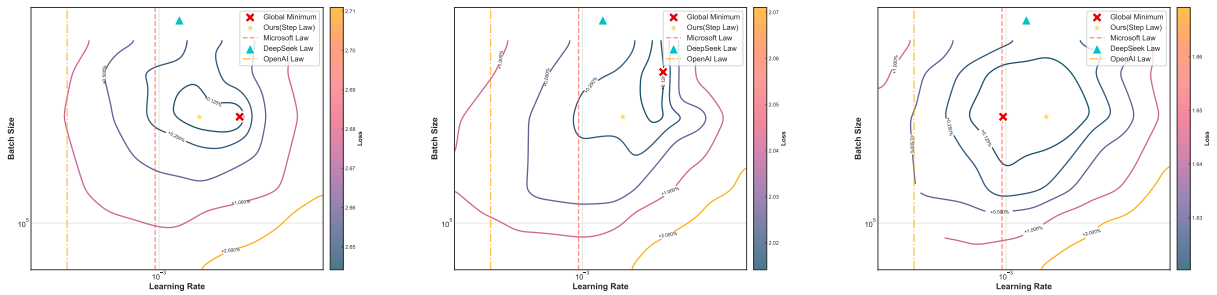


Figure 8: **Validation loss landscapes of MoE models under varying sparsity ratios (N_a/N).** Left: Low sparsity ($N_a/N = 0.27$). Middle: Medium sparsity ($N_a/N = 0.58$). Right: Medium sparsity at D=8.0B. Our method consistently approximates global minima across sparsity regimes.



(a) Bilingual Corpus (b) Code Integration (c) Code-Dominante

Figure 9: **Configuration Space Analysis under Different Data Recipes.** Our method demonstrates stable convergence patterns across varying data compositions.

- The HP scaling law demonstrates statistical invariance across linguistic and structural changes in the dataset, supporting its generalizability beyond standard natural language distributions.

- The predicted optimal hyperparameters remain stable even with highly heterogeneous training data, reinforcing our approach's robustness.

These findings are particularly significant for designing scalable and adaptable training paradigms applicable across diverse deployment scenarios with varying dataset characteristics.

Key Takeaways

- **Topological Invariance:** Our HP scaling laws exhibit statistical invariance in the scaling constants for LR and BS with respect to model scale N and data size D, even when varying topological features of model architectures.

- **Sparsity Independence:** The HP scaling law extends beyond dense Transformers and remains effective for sparse MoE models, with our approach achieving superior prediction accuracy across different sparsity levels, reinforcing the broader applicability of scaling laws in diverse neural architectures $\mathbb{A}$.

- **Data-Distribution Robustness:** The HP scaling law shows robustness across diverse data distributions $\mathbb{D}$.

6 Conclusions

In this paper, we provide a crucial advancement in efficient hyperparameter optimization for LLMs. By empirically unveiling and rigorously validating universal scaling laws for learning rate and batch size—underpinned by the discovery of loss landscape convexity—we move beyond computationally expensive grid searches and limited transfer methods. Our robust HP scaling laws, supported by an unprecedentedly large empirical study and open-sourced resources, empower the community with a practical and generalizable approach for navigating the hyperparameter configuration space in LLM pretraining, thereby facilitating more efficient and scalable LLM development.

Limitations

While our empirical study provides valuable universal HP scaling laws and demonstrates their practical efficacy, it is essential to acknowledge the limitations inherent in an empirical approach. Our findings are primarily data-driven. Future work should focus on developing a more theoretical understanding of the observed power-law relationships, potentially deriving them from first principles to enhance their predictive power and generalizability beyond the empirically validated domain.

Acknowledgments

The work was supported by National Science and Technology Major Project of China (2023ZD0121300).

References

Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. 2023. Pythia: A suite for analyzing large language models across training and scaling.

Johan Bjorck, Alon Benhaim, Vishrav Chaudhary, Furu Wei, and Xia Song. 2024. Scaling optimal lr across token horizons.

Charlie Blake, Constantin Eichenberg, Josef Dean, Lukas Balles, Luke Y. Prince, Björn Deiseroth, Andres Felipe Cruz-Salinas, Carlo Luschi, Samuel Weinbach, and Douglas Orr. 2024. u-μp: The unit-scaled maximal update parametrization.

Blake Bordelon, Lorenzo Noci, Mufan Bill Li, Boris Hanin, and Cengiz Pehlevan. 2023. Depthwise hyperparameter transfer in residual networks: Dynamics and scaling limit.

Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners.

DeepSeek-AI, Xiao Bi, Deli Chen, Guanting Chen, Shanhuang Chen, Damai Dai, Chengqi Deng, and et al. 2024a. Deepseek llm: Scaling open-source language models with longtermism.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, et al. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning.

DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024b. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*.

Nan Du, Yanping Huang, Andrew M. Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten Bosma, Zongwei Zhou, Tao Wang, Yu Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc V Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. 2021. Glam: Efficient scaling of language models with mixture-of-experts.

Katie Everett, Lechao Xiao, Mitchell Wortsman, Alexander A. Alemi, Roman Novak, Peter J. Liu, Izzeddin Gur, Jascha Sohl-Dickstein, Leslie Pack Kaelbling, Jaehoon Lee, and Jeffrey Pennington. 2024. Scaling exponents across parameterizations and optimizers.

William Fedus, Barret Zoph, and Noam Shazeer. 2021. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity.

William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39.

Oleg Filatov, Jan Ebert, Jiangtao Wang, and Stefan Kesselheim. 2024. Time transfer: On optimal learning rate and batch size in the infinite data limit.

Philip Gage. 1994. A new algorithm for data compression. *The C Users Journal archive*, 12:23–38.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, and Amy Yang et al. 2024. The llama 3 herd of models.

Alon Halfon, Shai Gretz, Ofir Arviv, Artem Spector, Orith Toledo-Ronen, Yoav Katz, Liat Ein-Dor, Michal Shmueli-Scheuer, and Noam Slonim. 2024. Stay tuned: An empirical study of the impact of hyperparameters on llm tuning in real-world applications.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack W. Rae, Oriol Vinyals, and Laurent Sifre. 2022. Training Compute-Optimal Large Language Models. ArXiv:2203.15556 [cs].

Shengding Hu, Yuge Tu, Xu Han, Chaoqun He, Ganqu Cui, Xiang Long, Zhi Zheng, Yewei Fang, Yuxiang Huang, Weilin Zhao, Xinrong Zhang, Zheng Leng Thai, Kaihuo Zhang, Chongyi Wang, Yuan Yao, Chenyang Zhao, Jie Zhou, Jie Cai, Zhongwu Zhai, Ning Ding, Chao Jia, Guoyang Zeng, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. Minicpm: Unveiling the potential of small language models with scalable training strategies.

Hongpeng Jin, Wenqi Wei, Xuyu Wang, Wenbin Zhang, and Yanzhao Wu. 2023. Rethinking learning rate tuning in the era of large language models.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling Laws for Neural Language Models. ArXiv:2001.08361 [cs, stat].

Lucas Lingle. 2024. A large-scale exploration of μ-transfer.

Ilya Loshchilov and Frank Hutter. 2017. Decoupled weight decay regularization.

Jan Ludziejewski, Maciej Pióro, Jakub Krajewski, Maciej Stefaniak, Michał Krutul, Jan Małaśnicki, Marek Cygan, Piotr Sankowski, Kamil Adamczewski, Piotr Miłoś, and Sebastian Jaszczur. 2025. Joint moe scaling laws: Mixture of experts can be memory efficient.

Sam McCandlish, Jared Kaplan, Dario Amodei, and OpenAI Dota Team. 2018. An empirical model of large-batch training.

Stefan Perko. 2023. Unlocking optimal batch size schedules using continuous-time control and perturbation theory.

Tomer Porian, Mitchell Wortsman, Jenia Jitsev, Ludwig Schmidt, and Yair Carmon. 2024. Resolving discrepancies in compute-optimal scaling of language models.

Ofir Press, Noah A. Smith, and Mike Lewis. 2021. Train short, test long: Attention with linear biases enables input length extrapolation.

Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, Thomas Wolf, et al. 2022. Bloom: A 176b-parameter open-access multilingual language model.

Noam Shazeer. 2020. Glu variants improve transformer.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*.

Yikang Shen, Matthew Stallone, Mayank Mishra, Gaoyuan Zhang, Shawn Tan, Aditya Prasad, Adriana Meza Soria, David D. Cox, and Rameswar Panda. 2024. Power scheduler: A batch size and token number agnostic learning rate scheduler.

Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. 2014. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023a. Llama: Open and efficient foundation language models.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, and Shruti Bhosale et al. 2023b. Llama 2: Open foundation and fine-tuned chat models.

Qiufeng Wang, Minghuan Liu, Weijia Zhang, Yuhang Guo, and Tianrui Li. 2019. Automatic proofreading in chinese: Detect and correct spelling errors in character-level with deep neural networks. In *Natural Language Processing and Chinese Computing: 8th CCF International Conference, NLPCC 2019, Dunhuang, China, October 9–14, 2019, Proceedings, Part II 8*, pages 349–359. Springer.

Siqi Wang, Zhengyu Chen, Bei Li, Keqing He, Min Zhang, and Jingang Wang. 2024. Scaling laws across model architectures: A comparative analysis of dense and moe models in large language models. pages 5583–5595.

Kaiyue Wen, Zhiyuan Li, Jason Wang, David Hall, Percy Liang, and Tengyu Ma. 2024. Understanding warmup-stable-decay learning rates: A river valley loss landscape perspective.

An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Qwen Team, and Alibaba Group et al. 2024. Qwen2 technical report.

Greg Yang and Edward J. Hu. 2020. Feature learning in infinite-width neural networks.

Greg Yang, Edward J. Hu, Igor Babuschkin, Szymon Sidor, Xiaodong Liu, David Farhi, Nick Ryder, Jakub Pachocki, Weizhu Chen, and Jianfeng Gao. 2022. Tensor programs v: Tuning large neural networks via zero-shot hyperparameter transfer.

Greg Yang, Dingli Yu, Chen Zhu, and Soufiane Hayou. 2023. Tensor programs vi: Feature learning in infinite-depth neural networks.

Wei Yuan, Hongzhi Yin, Tieke He, Tong Chen, Qiufeng Wang, and Lizhen Cui. 2022. Unified question generation with continual lifelong learning. In *Proceedings of the ACM Web Conference 2022*, pages 871–881.

Biao Zhang and Rico Sennrich. 2019. Root mean square layer normalization.

Hanlin Zhang, Depen Morwani, Nikhil Vyas, Jingfeng Wu, Difan Zou, Udaya Ghai, Dean Foster, and Sham Kakade. 2024. How does critical batch size scale in pre-training?

A Appendix

A.1 Model Scale Dominates Optimal Hyperparameter Selection Over Computational Complexity

To investigate how model architecture variations affect optimal hyperparameter settings, we conducted two sets of control experiments. In the first set, we maintained a constant parameter count (N), while in the second set, we kept the computational complexity (M) constant. Both sets used identical training configurations with 8B training tokens, varying only in their architectural proportions.

Tab. 3 presents the detailed configurations and results for both experimental groups. For each model, we systematically varied the hidden dimension (d_{model}), feed-forward dimension (d_{ff}), number of attention heads (N_{head}), and number of layers (N_{layer}) while maintaining either constant N or M. The embedding dimension (D) was fixed at 8.00E+09 across all experiments.

To visualize the impact of hyperparameters across different architectural configurations, we generated heatmaps of the loss landscape with respect to LR and BS in Fig. 7 and 10. The heatmaps reveal consistent patterns in the optimal hyperparameter regions across different architectural configurations within each experimental group.

The experimental results reveal several key findings: (i) Models with constant N demonstrate remarkably consistent optimal hyperparameter regions, with minimal variation in minimum loss values (ranging from 2.4294 to 2.4776) despite significant architectural differences. (ii) The constant M experiments show slightly more variation in optimal hyperparameter regions and minimum loss values (ranging from 2.4346 to 2.5089), suggesting that parameter count N may be a more robust indicator for hyperparameter selection than computational complexity M. (iii) Across both experimental groups, the optimal learning rates typically

d_{model}	d_{ff}	N_{head}	N_{layer}	lr	bs	D	N	M
Constant N Experiments								
1280	12264	10	8	1.95E-03	262,144	8.00E+09	4.29E+08	2.83E+09
1280	6280	10	14	1.38E-03	262,144	8.00E+09	4.29E+08	3.02E+09
1536	9600	12	8	9.77E-04	131,072	8.00E+09	4.29E+08	2.88E+09
1536	7264	12	10	1.38E-03	262,144	8.00E+09	4.29E+08	2.95E+09
1536	4608	12	14	9.77E-04	131,072	8.00E+09	4.29E+08	3.10E+09
2048	6000	16	8	9.77E-04	262,144	8.00E+09	4.29E+08	2.98E+09
2048	4256	16	10	9.77E-04	262,144	8.00E+09	4.29E+08	3.08E+09
2048	2256	16	14	9.77E-04	262,144	8.00E+09	4.29E+08	3.28E+09
Constant M Experiments								
1280	12608	10	8	1.38E-03	262,144	8.00E+09	4.40E+08	2.89E+09
1280	5888	10	14	1.38E-03	262,144	8.00E+09	4.08E+08	2.89E+09
1536	9656	12	8	1.38E-03	262,144	8.00E+09	4.31E+08	2.89E+09
1536	7040	12	10	1.38E-03	262,144	8.00E+09	4.19E+08	2.89E+09
1536	4056	12	14	9.77E-04	262,144	8.00E+09	3.94E+08	2.89E+09
2048	5704	16	8	9.77E-04	262,144	8.00E+09	4.15E+08	2.89E+09
2048	3744	16	10	6.91E-04	131,072	8.00E+09	3.98E+08	2.89E+09
2048	1504	16	14	6.91E-04	131,072	8.00E+09	3.64E+08	2.89E+09

Table 3: **Model configurations for constant N and constant M experiments.** The first group (top) maintains constant parameter count $N \approx 4.29\text{E+}08$, while the second group (bottom) maintains constant computational complexity $M \approx 2.89\text{E+}09$. M: non-embedding FLOPs/token.

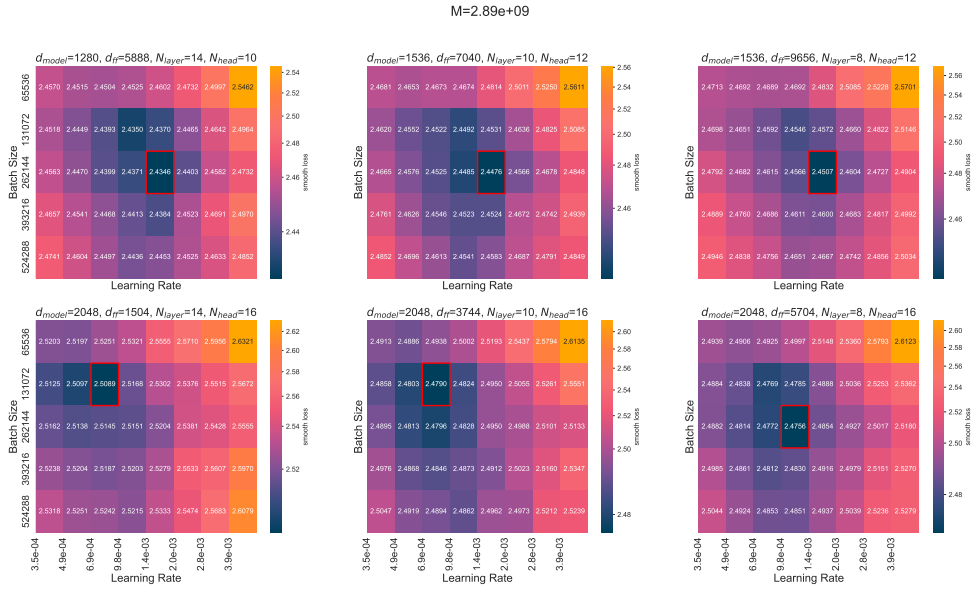


Figure 10: Loss landscapes visualized as heatmaps across learning rate (x-axis) and batch size (y-axis) configurations. Darker colors indicate lower loss values. Shows results for models with constant computational complexity M, exhibiting slightly more variance in optimal hyperparameter regions.

fall within a narrow range (6.91E-04 to 1.95E-03), and batch sizes cluster around either 131,072 or 262,144, regardless of the specific architectural configuration.

These findings strongly suggest that the fundamental scale metrics, particularly the parameter count N, are more influential in determining optimal hyperparameter settings than specific architectural choices. This observation motivates our discussion of hyperparameter scaling laws in rela-

tion to N in Section 4.2.

A.2 Model Structural Parameters And Results

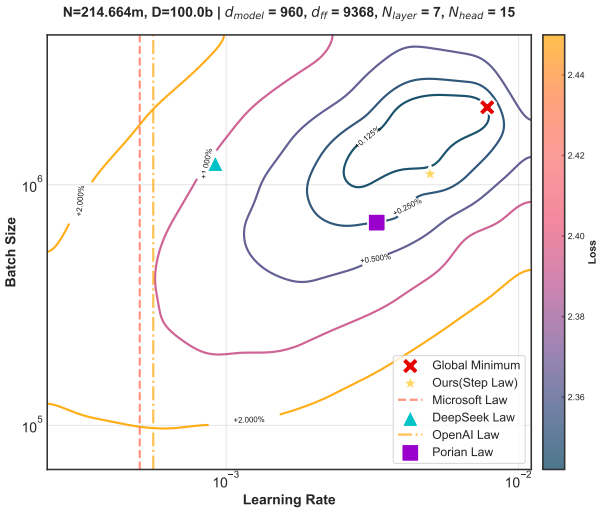


Figure 11: Illustration of Hyperparameter Configuration Space for Model 0.

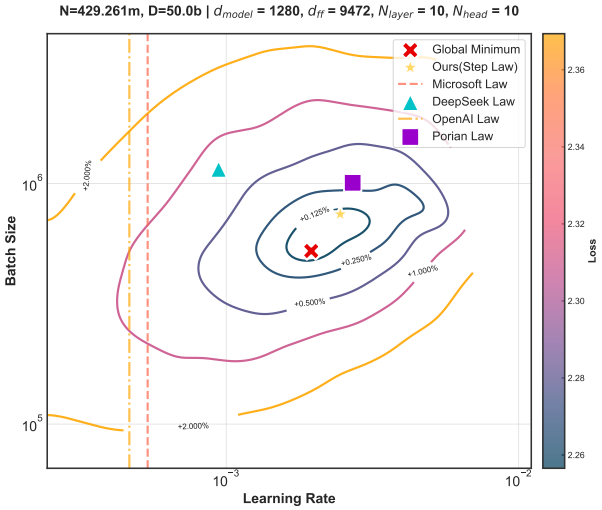


Figure 12: Illustration of Hyperparameter Configuration Space for Model 1.

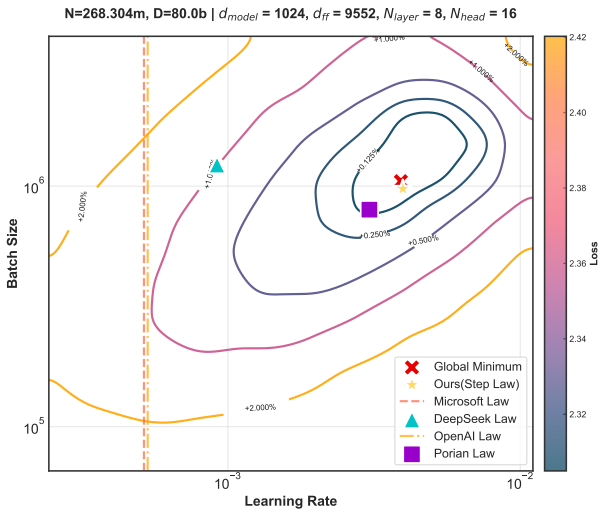


Figure 13: Illustration of Hyperparameter Configuration Space for Model 2.

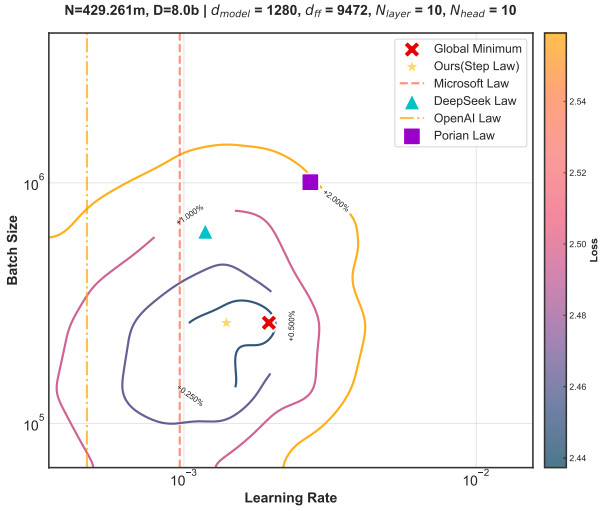


Figure 14: Illustration of Hyperparameter Configuration Space for Model 3.

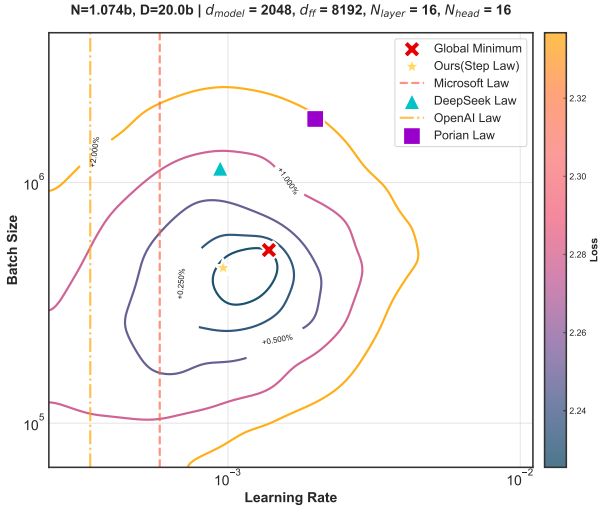


Figure 15: Illustration of Hyperparameter Configuration Space for Model 4.

Model	N	D	d_{model}	d_{ff}	N_{head}	N_{layer}
0	2.15E+08	1.14E+10	960	9368	15	7
1	4.29E+08	5.00E+10	1280	9472	10	10
2	2.68E+08	8.00E+10	1024	9552	16	8
3	4.29E+08	8.00E+09	1280	9472	10	10
4	1.07E+09	2.00E+10	2048	8192	16	16
5	5.37E+08	1.00E+10	1280	9048	10	13
6	2.15E+08	4.00E+09	960	9368	15	7
7	2.68E+08	5.00E+09	1024	9552	16	8
8	2.68E+08	1.42E+10	1024	9552	16	8
9	1.07E+09	5.69E+10	2048	8192	16	16
10	2.15E+08	1.00E+11	960	9368	15	7
11	4.29E+08	2.27E+10	1280	9472	10	10
12	5.37E+08	2.84E+10	1280	9048	10	13
13	2.15E+08	2.00E+10	960	9368	15	7
14	4.29E+08	4.00E+10	1280	9472	10	10
15	2.68E+08	2.50E+10	1024	9552	16	8
16	5.37E+08	5.00E+10	1280	9048	10	13
17	1.07E+09	1.00E+11	2048	8192	16	16

Table 4: **Dense Model Configuration.**

Model	N	D	d_{model}	N_{head}	N_{layer}	N_{expert}	d_{moe}	Top-k	N_a
0	2150612992	2000000000	1408	11	16	89	352	1	187973632
1	2150612992	2000000000	1408	11	16	88	352	2	232579072
2	2155174912	2000000000	1408	11	16	8	3528	1	590436352
3	2156188672	2000000000	1408	11	16	8	2888	3	1241270272
4	2150612992	4000000000	1408	11	16	89	352	1	187973632
5	2150612992	4000000000	1408	11	16	88	352	2	232579072
6	2155174912	4000000000	1408	11	16	8	3528	1	590436352
7	2156188672	4000000000	1408	11	16	8	2888	3	1241270272
8	2150612992	8000000000	1408	11	16	89	352	1	187973632
9	2150612992	8000000000	1408	11	16	88	352	2	232579072
10	2155174912	8000000000	1408	11	16	8	3528	1	590436352
11	2156188672	8000000000	1408	11	16	8	2888	3	1241270272
12	2150612992	20000000000	1408	11	16	89	352	1	187973632
13	2150612992	20000000000	1408	11	16	88	352	2	232579072
14	2155174912	20000000000	1408	11	16	8	3528	1	590436352
15	2156188672	20000000000	1408	11	16	8	2888	3	1241270272

Table 5: **MoE Model Configuration.** N_{expert} denotes the number of experts. d_{moe} denotes the hidden size of experts. Top-k denotes the number in the routing algorithm. N_a denotes the activate parameters.

Dataset	Baseline	Code+Math	More Code+Math	En-CN
web-data-en	79.53	44.75	20.00	44.99
web-data-cn	–	–	–	34.52
code-the-stack	4.62	32.36	57.05	4.63
web-data-math	–	7.07	7.07	–
book-non-novel-en	4.35	4.34	4.34	4.35
paper	3.38	3.37	3.37	3.38
wikipedia-mtlg	3.24	3.24	3.24	3.25
stackexchange	2.21	2.21	2.21	2.22
wikipedia-en	1.69	1.69	1.69	1.69
book-novel-en	0.83	0.83	0.83	0.83
wikipedia-cn	0.13	0.13	0.13	0.13

Table 6: **Comparison of dataset weights (%) across different training recipes.** Each recipe represents a different focus: baseline, enhanced code and mathematics capability, and English-Chinese bilingual ability.

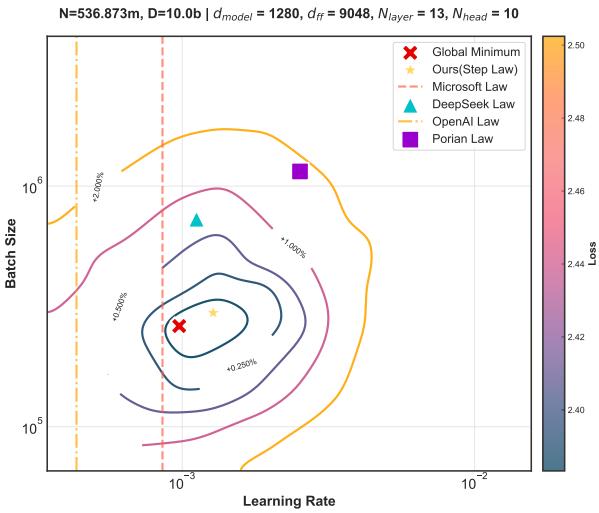


Figure 16: Illustration of Hyperparameter Configuration Space for Model 5.

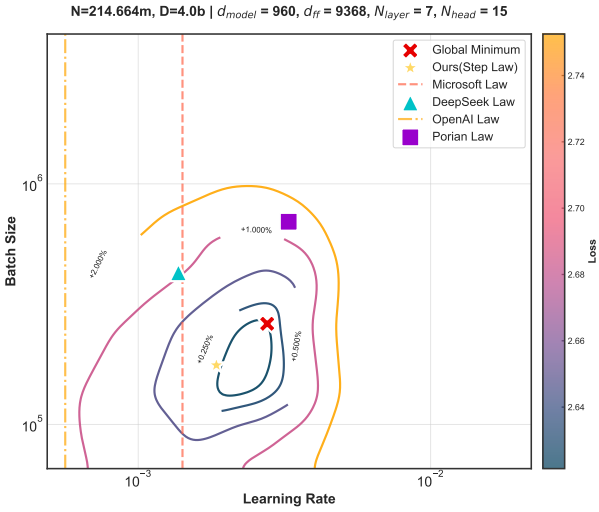


Figure 17: Illustration of Hyperparameter Configuration Space for Model 6.

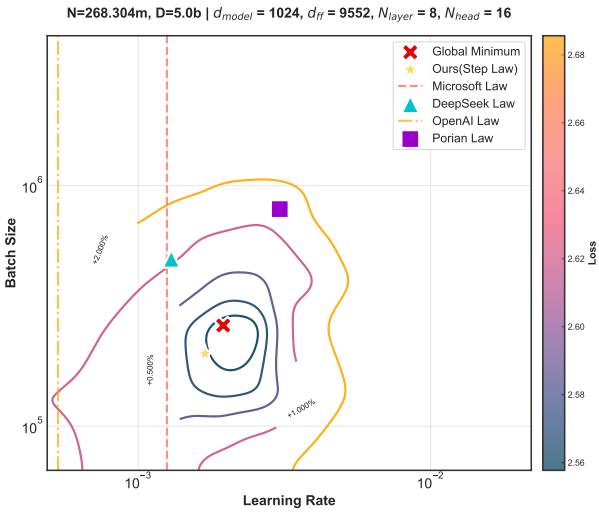


Figure 18: Illustration of Hyperparameter Configuration Space for Model 7.

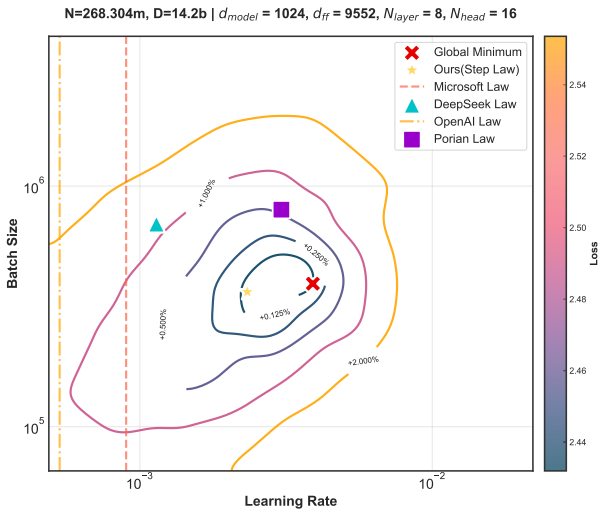


Figure 19: Illustration of Hyperparameter Configuration Space for Model 8.

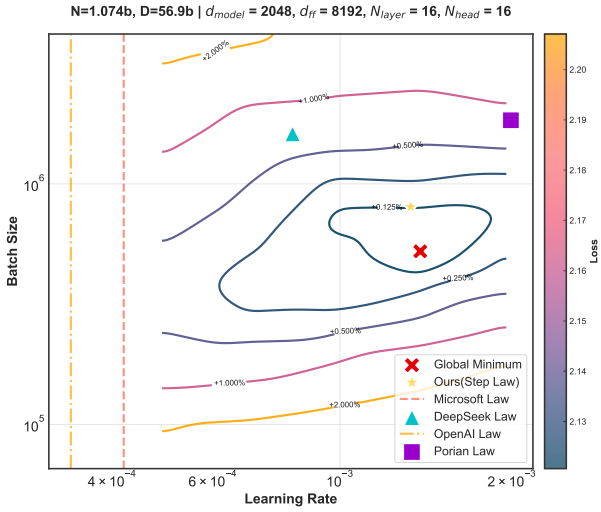


Figure 20: Illustration of Hyperparameter Configuration Space for Model 9.

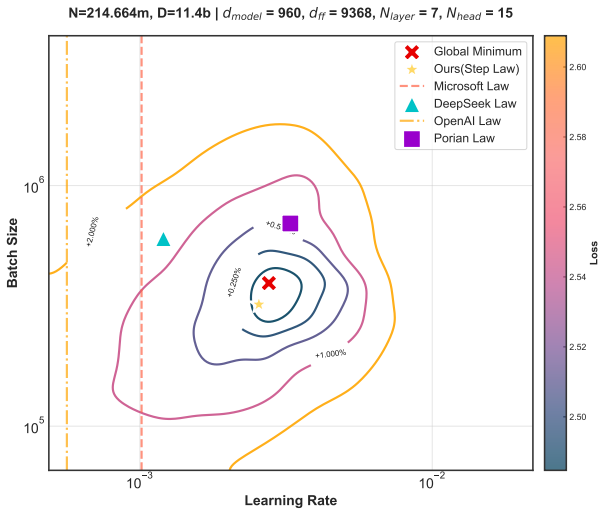


Figure 21: Illustration of Hyperparameter Configuration Space for Model 10.

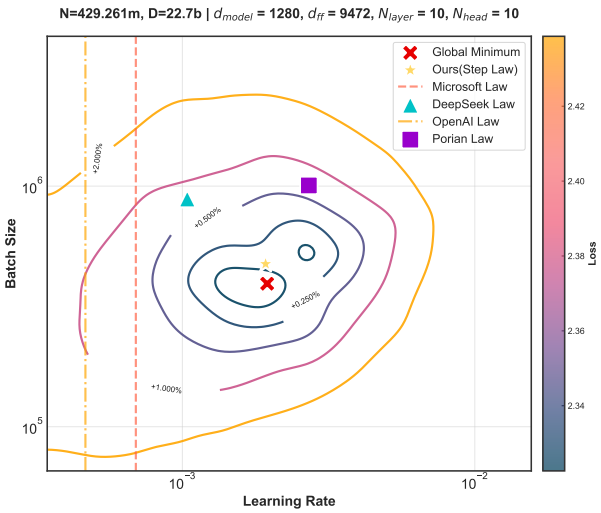


Figure 22: Illustration of Hyperparameter Configuration Space for Model 11.

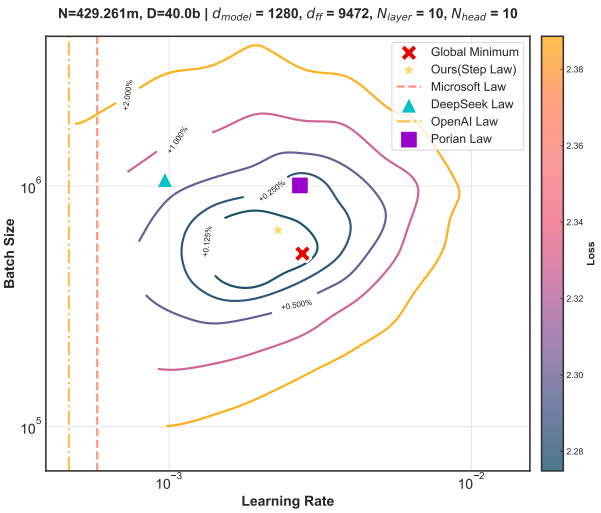


Figure 25: Illustration of Hyperparameter Configuration Space for Model 14.

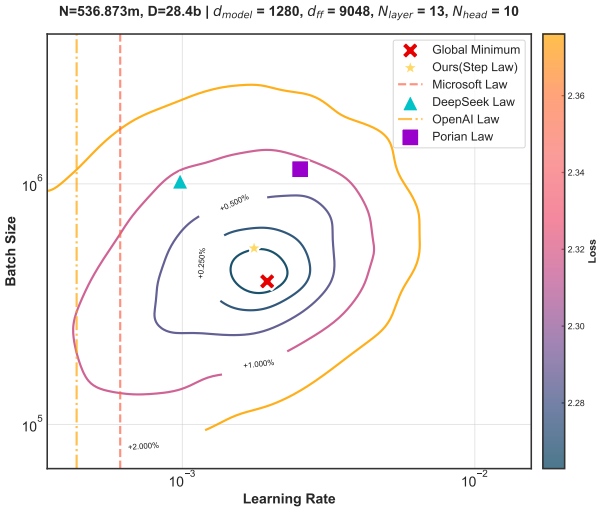


Figure 23: Illustration of Hyperparameter Configuration Space for Model 12.

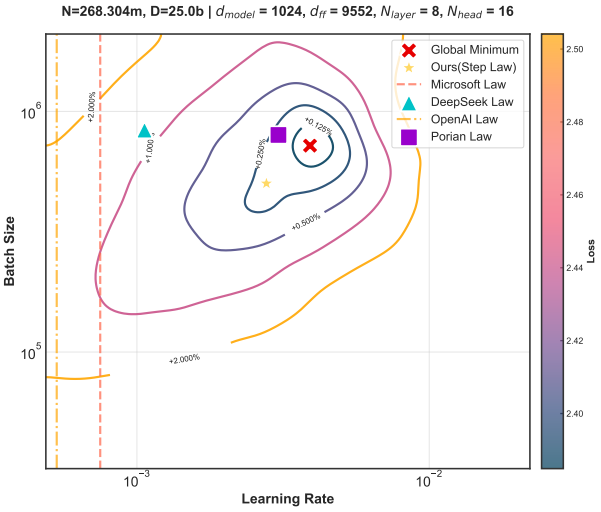


Figure 26: Illustration of Hyperparameter Configuration Space for Model 15.

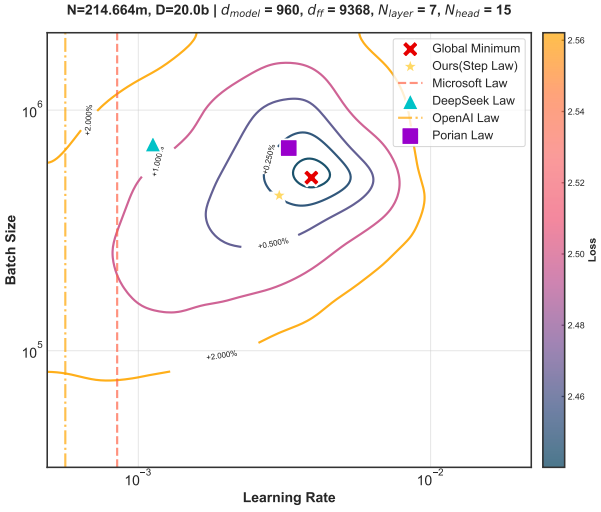


Figure 24: Illustration of Hyperparameter Configuration Space for Model 13.

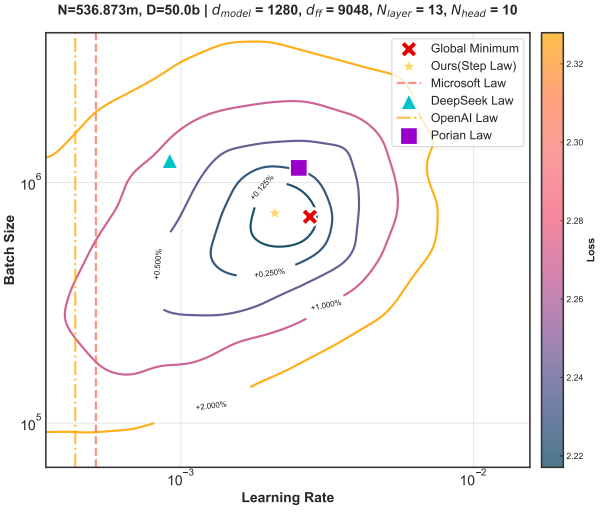


Figure 27: Illustration of Hyperparameter Configuration Space for Model 16.

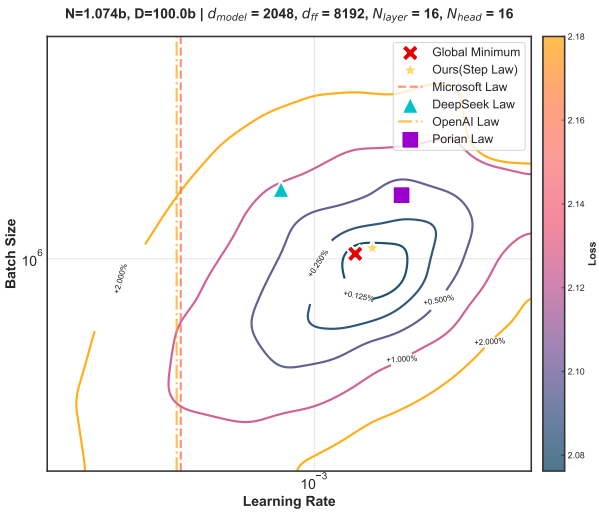


Figure 28: Illustration of Hyperparameter Configuration Space for Model 17.

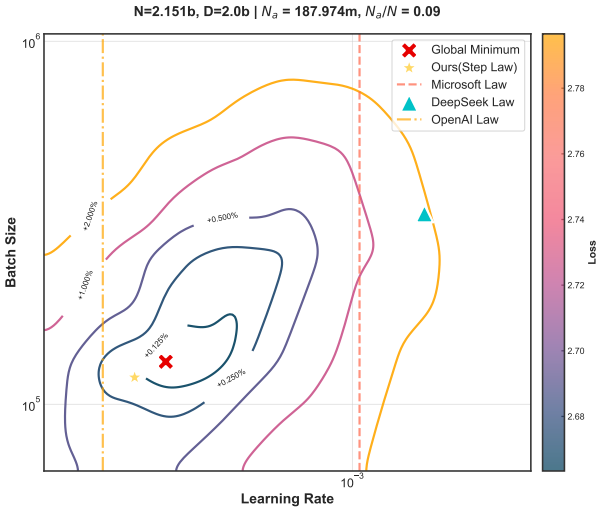


Figure 29: Illustration of Hyperparameter Configuration Space for MoE Model 0.

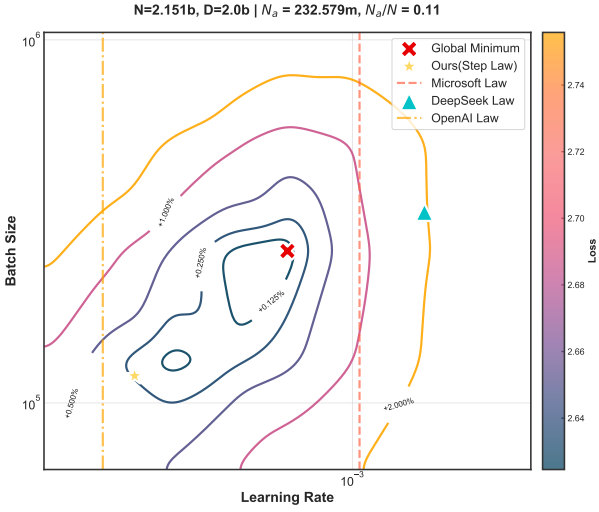


Figure 30: Illustration of Hyperparameter Configuration Space for MoE Model 1.

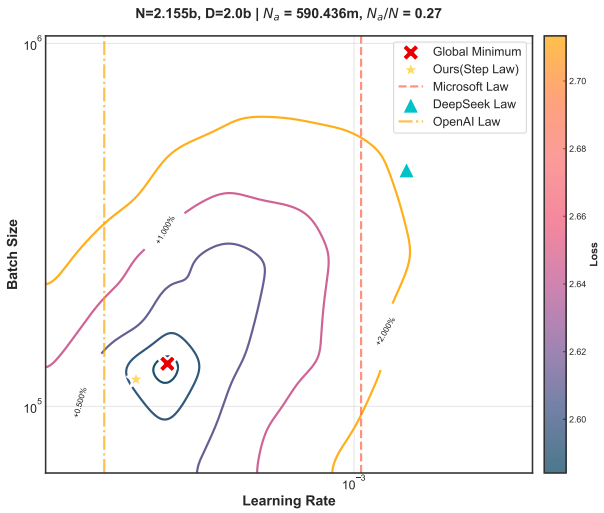


Figure 31: Illustration of Hyperparameter Configuration Space for MoE Model 2.

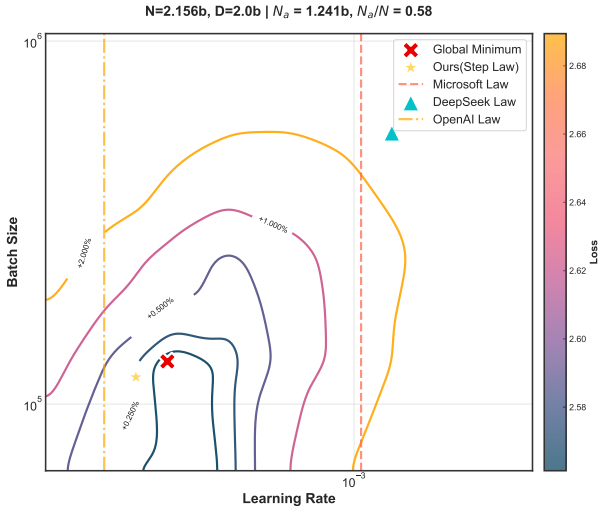


Figure 32: Illustration of Hyperparameter Configuration Space for MoE Model 3.

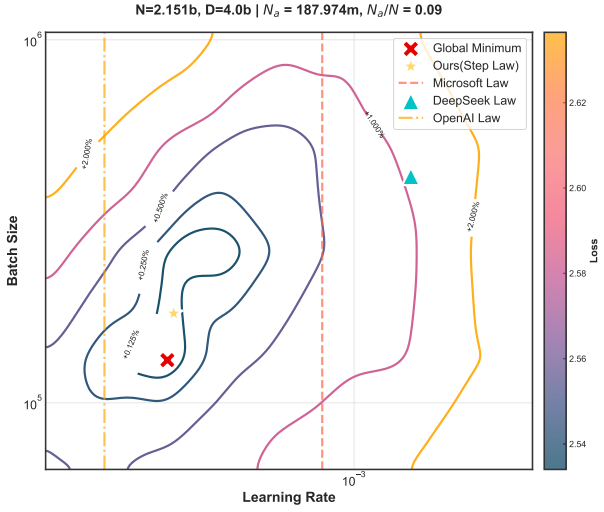


Figure 33: Illustration of Hyperparameter Configuration Space for MoE Model 4.

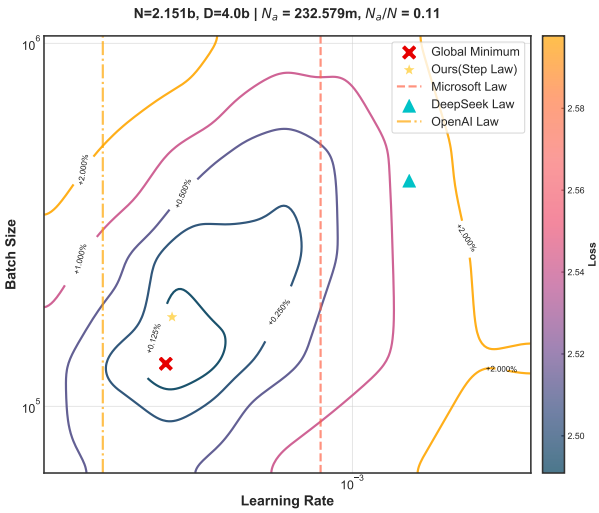


Figure 34: Illustration of Hyperparameter Configuration Space for MoE Model 5.

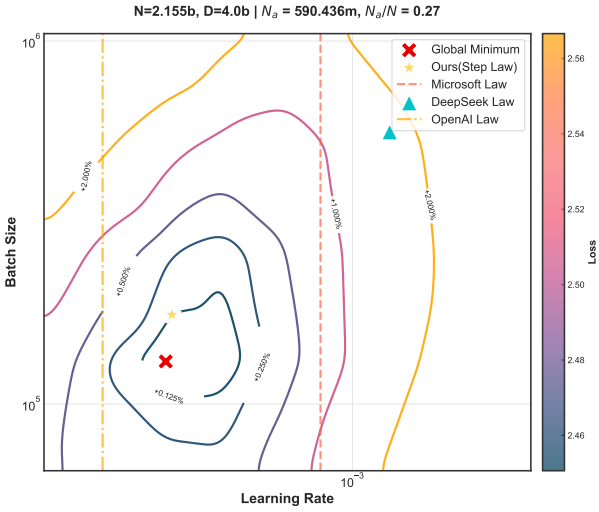


Figure 35: Illustration of Hyperparameter Configuration Space for MoE Model 6.

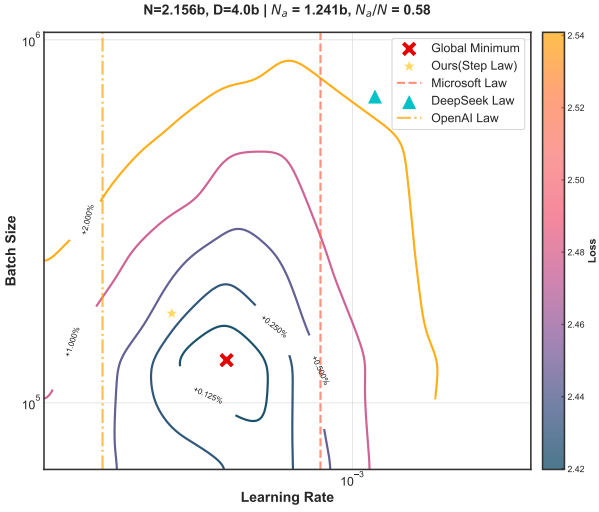


Figure 36: Illustration of Hyperparameter Configuration Space for MoE Model 7.

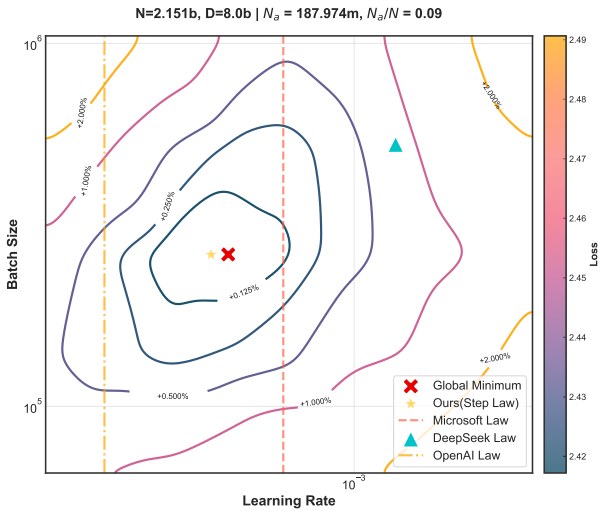


Figure 37: Illustration of Hyperparameter Configuration Space for MoE Model 8.

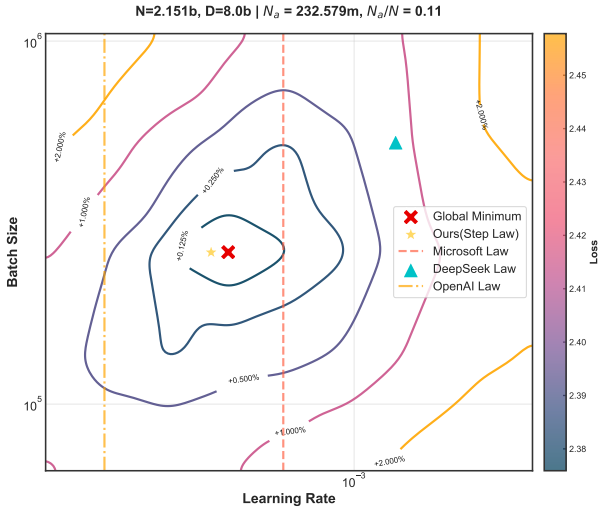


Figure 38: Illustration of Hyperparameter Configuration Space for MoE Model 9.

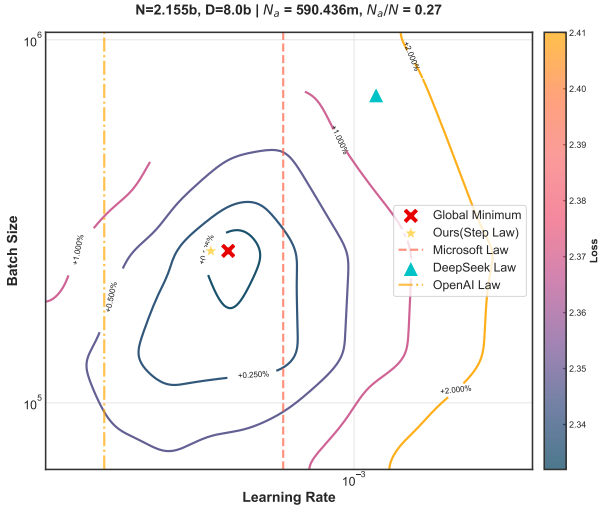


Figure 39: Illustration of Hyperparameter Configuration Space for MoE Model 10.

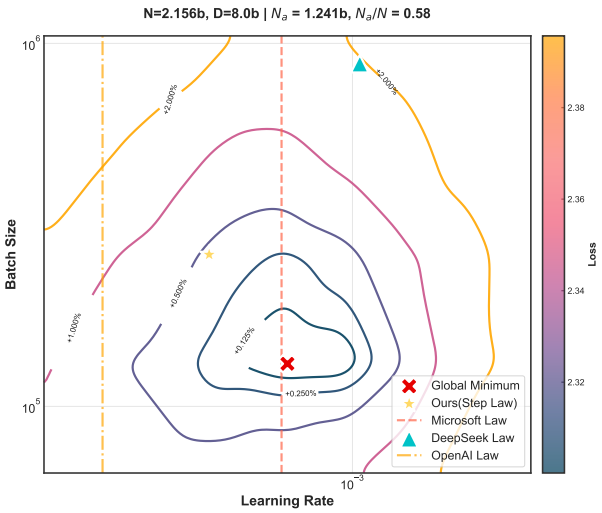


Figure 40: Illustration of Hyperparameter Configuration Space for MoE Model 11.

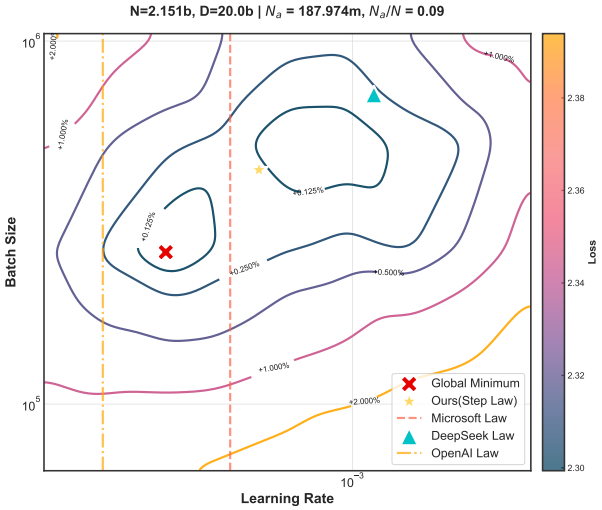


Figure 41: Illustration of Hyperparameter Configuration Space for MoE Model 12.

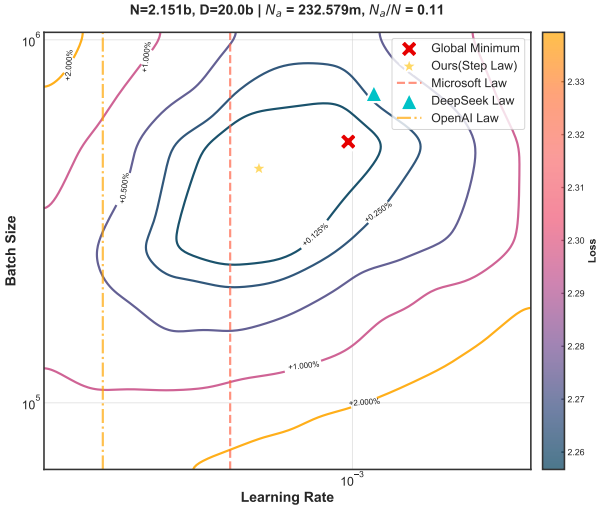


Figure 42: Illustration of Hyperparameter Configuration Space for MoE Model 13.

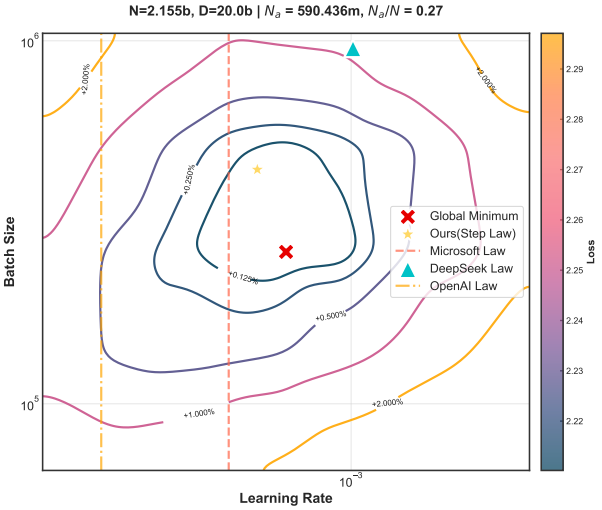


Figure 43: Illustration of Hyperparameter Configuration Space for MoE Model 14.

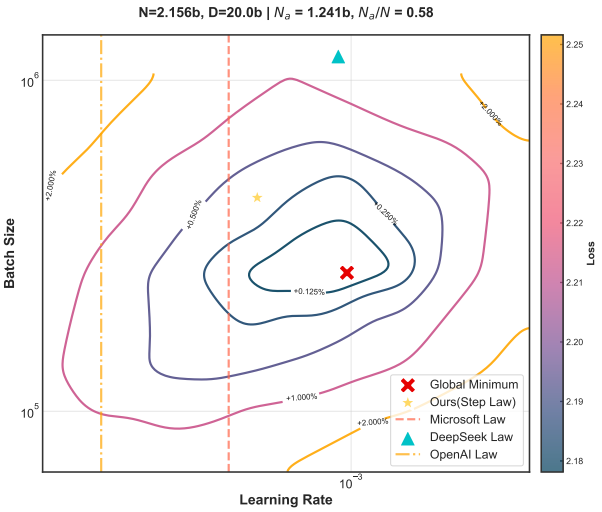


Figure 44: Illustration of Hyperparameter Configuration Space for MoE Model 15.